Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	7	Academic Year	:	2026-27

Task - 1 (Practical - 7)

Aim

To implement JWT-based authentication and input validation as part of the Express middleware pipeline, protecting task routes, hashing passwords securely using bcrypt, implementing a /me user profile endpoint, and handling token expiry gracefully on the frontend.

Code

1. server/middleware/auth.js (JWT Bearer Verification Middleware):

```
const jwt = require('jsonwebtoken');

/**
 * Authentication Middleware (Practical 7)
 * Verifies JWT token supplied in Authorization header (Bearer <token>)
 * Attaches decoded payload (user id, etc.) to req.user
 */
const authMiddleware = (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader) {
      return res.status(401).json({
        success: false,
        error: 'Authorization header is missing. Access denied.'
      });
    }

    const parts = authHeader.split(' ');
    if (parts.length !== 2 || parts[0] !== 'Bearer') {
      return res.status(401).json({
        success: false,
        error: 'Invalid token format. Expected: Bearer <token>'
      });
    }

    const token = parts[1];
    if (!token) {
      return res.status(401).json({
        success: false,
        error: 'Authentication token is missing. Access denied.'
      });
    }

    const secret = process.env.JWT_SECRET || 'fallback_secret_for_tests';
```

```
    const decoded = jwt.verify(token, secret);
    req.user = decoded;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Token has expired. Please log in again.'
      });
    }
    return res.status(401).json({
      success: false,
      error: 'Invalid token. Verification failed.'
    });
  }
};

module.exports = authMiddleware;
```

2. server/middleware/validate.js (Server-Side Input Validation Pipeline):

```
const mongoose = require('mongoose');

/**
 * Server-side Input Validation Middleware Pipeline (Practical 7)
 * Intercepts requests and enforces field validation before reaching controllers.
 */

// Validate user registration input
const validateRegister = (req, res, next) => {
  const { email, password } = req.body || {};

  if (!email || typeof email !== 'string' || !email.trim()) {
    return res.status(400).json({
      success: false,
      error: 'Email is required and cannot be empty.'
    });
  }

  const emailRegex = /^\S+@\S+\.\S+$/;
  if (!emailRegex.test(email.trim())) {
    return res.status(400).json({
      success: false,
      error: 'Please provide a valid email address.'
    });
  }

  if (!password || typeof password !== 'string') {
    return res.status(400).json({
      success: false,
      error: 'Password is required.'
    });
  }
```

```
  if (password.length < 6) {
    return res.status(400).json({
      success: false,
      error: 'Password must be at least 6 characters long.'
    });
  }

  next();
};

// Validate user login input
const validateLogin = (req, res, next) => {
  const { email, password } = req.body || {};

  if (!email || typeof email !== 'string' || !email.trim()) {
    return res.status(400).json({
      success: false,
      error: 'Email is required.'
    });
  }

  if (!password || typeof password !== 'string') {
    return res.status(400).json({
      success: false,
      error: 'Password is required.'
    });
  }

  next();
};

// Validate task creation and update input
const validateTask = (req, res, next) => {
  const { title, priority } = req.body || {};

  // For POST creation, title is strictly required
  if (req.method === 'POST') {
    if (!title || typeof title !== 'string' || !title.trim()) {
      return res.status(400).json({
        success: false,
        error: 'Task title is required and cannot be empty.'
      });
    }
  }

  // For PUT update, if title is provided, ensure it's not empty string
  if (req.method === 'PUT' && title !== undefined) {
    if (typeof title !== 'string' || !title.trim()) {
      return res.status(400).json({
        success: false,
        error: 'Task title cannot be empty.'
      });
    }
  }
```

```
  // Validate priority if provided
  if (priority !== undefined) {
    const validPriorities = ['low', 'medium', 'high'];
    if (typeof priority !== 'string' || !validPriorities.includes(priority.toLowerCase())) {
      return res.status(400).json({
        success: false,
        error: "Priority must be one of: 'low', 'medium', 'high'."
      });
    }
  }

  next();
};

// Validate 24-character hex MongoDB ObjectId
const validateObjectId = (req, res, next) => {
  const { id } = req.params;
  if (!id || id.length !== 24 || !mongoose.Types.ObjectId.isValid(id)) {
    return res.status(400).json({
      success: false,
      error: `Invalid task ID format: '${id}'. Must be a 24-character hexadecimal ObjectId.`
    });
  }
  next();
};

module.exports = {
  validateRegister,
  validateLogin,
  validateTask,
  validateObjectId
};
```

3. server/models/User.js (User Schema & Validation Rules):

```
const mongoose = require('mongoose');

/**
 * User Schema Definition (Practical 7)
 * Fields:
 * - email: String, required, unique, lowercase, trimmed
 * - password: String (hashed via bcrypt), required
 * - name: String, optional displayName
 * - createdAt: Date, default Date.now
 */
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    trim: true,
    lowercase: true,
```

```
    match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters long']
  },
  name: {
    type: String,
    trim: true,
    default: ''
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('User', userSchema);
```

4. server/models/Task.js (Task Schema with User Scoping & Pre-Save Hook):

```
const mongoose = require('mongoose');

/**
 * Task Schema Definition (Practical 5, 6, 7)
 * Fields:
 * - title: String, required
 * - description: String
 * - completed: Boolean, default false
 * - priority: String, enum ['low', 'medium', 'high'], default 'medium'
 * - user: ObjectId referencing User model (Practical 7 user scoping)
 * - createdAt: Date, default Date.now
 */
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Task title is required'],
    trim: true
  },
  description: {
    type: String,
    default: '',
    trim: true
  },
  completed: {
    type: Boolean,
    default: false
  },
  priority: {
    type: String,
    enum: {
      values: ['low', 'medium', 'high'],
```

```
      message: 'Priority must be one of: low, medium, high'
    },
    default: 'medium',
    lowercase: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: false
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Pre-save hook that automatically trims whitespace from the title field
taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

5. server/server.js (Express Server with Auth Pipeline, Protected Routes & Error Interceptors):

```
/**
 * Charotar University of Science and Technology (CHARUSAT)
 * Faculty of Technology and Engineering (FTE / CSPIT)
 * Course: Advanced Web Development Frameworks (ITUE301)
 * Practical 7: Authentication and Middleware Pipeline
 * Student: Manav Lakhani (24CS047)
 */

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const User = require('./models/User');
const Task = require('./models/Task');
const authMiddleware = require('./middleware/auth');
const {
  validateRegister,
  validateLogin,
  validateTask,
  validateObjectId
} = require('./middleware/validate');

const app = express();
const PORT = process.env.PORT || 5000;
```

```
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_manager_db';
const JWT_SECRET = process.env.JWT_SECRET || 'jay_ladva_24cs047_jwt_default_secret_2026';

// ========================================
// 1. CORS Middleware
// ========================================
app.use(cors({
  origin: ['http://localhost:5173', 'http://127.0.0.1:5173'],
  credentials: true,
  allowedHeaders: ['Content-Type', 'Authorization']
}));

// ========================================
// 2. Built-in JSON Body Parsing Middleware
// ========================================
app.use(express.json());

// ========================================
// 3. Global Request Logging Middleware
// ========================================
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

// ========================================
// 4. Content-Type Header Enforcement Middleware
// ========================================
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.toLowerCase().includes('application/json')) {
      return res.status(400).json({
        success: false,
        error: 'Bad Request',
        message: 'Requests with payload (POST/PUT) must include "Content-Type: application/json"
header.'
      });
    }
  }
  next();
};

app.use(requireJsonContent);

// ========================================
// 5. MongoDB Connection
// ========================================
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('[MongoDB] Connected successfully to:', MONGO_URI.replace(/:[^:]*@/, ':****@'));
  })
  .catch((err) => {
```

```
    console.error('[MongoDB] Connection error:', err.message);
  });

// ========================================
// 6. Base / Health Route
// ========================================
app.get('/', (req, res) => {
  const dbStatus = mongoose.connection.readyState === 1 ? 'connected' : 'disconnected';
  res.status(200).json({
    message: 'Welcome to Task Manager API with JWT Authentication (Practical 7)',
    student: 'Manav Lakhani (24CS047)',
    course: 'ITUE301',
    database: {
      type: 'MongoDB Atlas',
      status: dbStatus
    },
    authEndpoints: {
      register: 'POST /register',
      login: 'POST /login',
      me: 'GET /me'
    },
    taskEndpoints: {
      getAll: 'GET /tasks (Protected)',
      getById: 'GET /tasks/:id (Protected)',
      create: 'POST /tasks (Protected)',
      update: 'PUT /tasks/:id (Protected)',
      delete: 'DELETE /tasks/:id (Protected)'
    }
  });
});

// ========================================
// 7. Authentication Routes
// ========================================

/**
 * Step 3: POST /register
 * Hashes password using bcrypt (cost factor 10) before saving User
 */
app.post('/register', validateRegister, async (req, res, next) => {
  try {
    const { email, password, name } = req.body;

    const existingUser = await User.findOne({ email: email.toLowerCase().trim() });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        error: 'A user with this email address is already registered.'
      });
    }

    // Hash password with bcrypt
    const hashedPassword = await bcrypt.hash(password, 10);

    const user = await User.create({
```

```
      email: email.toLowerCase().trim(),
      password: hashedPassword,
      name: name ? name.trim() : ''
    });

    // Sign JWT token for automatic authentication upon registration
    const token = jwt.sign(
      { id: user._id, email: user.email },
      JWT_SECRET,
      { expiresIn: '1h' }
    );

    res.status(201).json({
      success: true,
      message: 'User registered successfully',
      token,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

/**
 * Step 4: POST /login
 * Compares plain password with bcrypt hash and signs JWT token (expires in 1h)
 */
app.post('/login', validateLogin, async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email: email.toLowerCase().trim() });
    if (!user) {
      return res.status(401).json({
        success: false,
        error: 'Invalid email or password.'
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({
        success: false,
        error: 'Invalid email or password.'
      });
    }

    const token = jwt.sign(
      { id: user._id, email: user.email },
      JWT_SECRET,
```

```
      { expiresIn: '1h' }
    );

    res.status(200).json({
      success: true,
      message: 'Login successful',
      token,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

/**
 * Supplementary Problem 1: GET /me
 * Returns currently logged-in user's details using decoded JWT
 */
app.get('/me', authMiddleware, async (req, res, next) => {
  try {
    const user = await User.findById(req.user.id).select('-password');
    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found.'
      });
    }

    res.status(200).json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        name: user.name,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
// 8. Protected Task Routes (Auth & Validation Pipeline)
// ========================================

/**
 * GET /tasks
 * Retrieves tasks for the authenticated user
 */
```

```
app.get('/tasks', authMiddleware, async (req, res, next) => {
  try {
    const { status, priority, search } = req.query;
    // Query tasks matching current user or unassigned legacy tasks
    const query = {
      $or: [{ user: req.user.id }, { user: { $exists: false } }]
    };

    if (status === 'completed') {
      query.completed = true;
    } else if (status === 'pending') {
      query.completed = false;
    }

    if (priority && ['low', 'medium', 'high'].includes(priority.toLowerCase())) {
      query.priority = priority.toLowerCase();
    }

    if (search && search.trim()) {
      query.$and = [
        {
          $or: [
            { title: { $regex: search.trim(), $options: 'i' } },
            { description: { $regex: search.trim(), $options: 'i' } }
          ]
        }
      ];
    }

    const tasks = await Task.find(query).sort({ createdAt: -1 });

    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

/**
 * GET /tasks/:id
 * Retrieves single task by ObjectId for the authenticated user
 */
app.get('/tasks/:id', authMiddleware, validateObjectId, async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);

    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task with ID '${req.params.id}' was not found.`
      });
    }
```

```
    res.status(200).json({
      success: true,
      data: task
    });
  } catch (err) {
    next(err);
  }
});

/**
 * POST /tasks
 * Creates task scoped to the authenticated user with validation pipeline
 */
app.post('/tasks', authMiddleware, validateTask, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    const newTask = new Task({
      title,
      description: description || '',
      completed: completed === true,
      priority: priority || 'medium',
      user: req.user.id
    });

    const savedTask = await newTask.save();

    res.status(201).json({
      success: true,
      message: 'Task created successfully',
      data: savedTask
    });
  } catch (err) {
    next(err);
  }
});

/**
 * PUT /tasks/:id
 * Updates task by ID with validation pipeline
 */
app.put('/tasks/:id', authMiddleware, validateObjectId, validateTask, async (req, res, next) => {
  try {
    const existingTask = await Task.findById(req.params.id);
    if (!existingTask) {
      return res.status(404).json({
        success: false,
        error: `Task with ID '${req.params.id}' was not found.`
      });
    }

    const updates = {};
    if (req.body.title !== undefined) updates.title = req.body.title;
    if (req.body.description !== undefined) updates.description = req.body.description;
```

```
    if (req.body.completed !== undefined) updates.completed = Boolean(req.body.completed);
    if (req.body.priority !== undefined) updates.priority = req.body.priority;

    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id,
      { $set: updates },
      { returnDocument: 'after', runValidators: true }
    );

    res.status(200).json({
      success: true,
      message: 'Task updated successfully',
      data: updatedTask
    });
  } catch (err) {
    next(err);
  }
});

/**
 * DELETE /tasks/:id
 * Deletes task by ID
 */
app.delete('/tasks/:id', authMiddleware, validateObjectId, async (req, res, next) => {
  try {
    const deletedTask = await Task.findByIdAndDelete(req.params.id);

    if (!deletedTask) {
      return res.status(404).json({
        success: false,
        error: `Task with ID '${req.params.id}' was not found.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully',
      data: deletedTask
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
// 9. Centralized Error Handling Middleware
// ========================================
app.use((err, req, res, next) => {
  console.error('[Error Pipeline]:', err);

  // Mongoose Validation Error (e.g., enum mismatch, missing required field)
  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map((val) => val.message);
    return res.status(400).json({
      success: false,
```

```
      error: 'Validation Error',
      details: messages
    });
  }

  // Mongoose CastError (e.g. invalid ObjectId)
  if (err.name === 'CastError') {
    return res.status(400).json({
      success: false,
      error: `Invalid ${err.path}: '${err.value}'`
    });
  }

  // Duplicate Key Error (e.g. unique email)
  if (err.code === 11000) {
    return res.status(400).json({
      success: false,
      error: 'Duplicate field value entered. Email must be unique.'
    });
  }

  // Fallback 500 Internal Server Error
  res.status(500).json({
    success: false,
    error: 'Internal Server Error',
    message: err.message || 'An unexpected error occurred on the server.'
  });
});

// ========================================
// 10. Fallback 404 Route Handler
// ========================================
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Not Found',
    message: `Cannot ${req.method} ${req.originalUrl}. Route not found.`
  });
});

// ========================================
// 11. Start Server
// ========================================
if (process.env.NODE_ENV !== 'test') {
  app.listen(PORT, () => {
    console.log(`[Practical 7 Server] Running on http://localhost:${PORT}`);
  });
}

module.exports = app;
```

6. client/src/api.js (Frontend API Client with JWT Bearer Injection & 401 Interceptor):

```
/**
 * Central API Client for Practical 7: Authentication and Middleware Pipeline
 * Communicates with the Express Backend (http://localhost:5000)
 * Handles JWT token storage, automatic Bearer Authorization injection,
 * and 401 Token Expiry interceptor.
 */

const BASE_URL = 'http://localhost:5000';

// Token Management
export const getToken = () => localStorage.getItem('token');
export const setToken = (token) => localStorage.setItem('token', token);
export const removeToken = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
};

export const getStoredUser = () => {
  try {
    const raw = localStorage.getItem('user');
    return raw ? JSON.parse(raw) : null;
  } catch (e) {
    return null;
  }
};

export const setStoredUser = (user) => {
  localStorage.setItem('user', JSON.stringify(user));
};

/**
 * Standard fetch wrapper that automatically attaches the Authorization header
 * and intercepts 401 Unauthorized responses to handle token expiry.
 */
const authFetch = async (endpoint, options = {}) => {
  const token = getToken();
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };

  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const response = await fetch(`${BASE_URL}${endpoint}`, {
    ...options,
    headers
  });

  // Supplementary Problem 2: Token expiry handling on the frontend
  if (response.status === 401) {
    removeToken();
    window.dispatchEvent(new CustomEvent('auth:unauthorized', {
      detail: { message: 'Session expired or unauthorized. Please log in again.' }
```

```
    }));
  }

  let data = null;
  try {
    data = await response.json();
  } catch (e) {
    data = { error: 'Invalid JSON response from server' };
  }

  if (!response.ok) {
    const errMsg = data.error || data.message || `Request failed with status ${response.status}`;
    const errorObj = new Error(errMsg);
    errorObj.status = response.status;
    errorObj.details = data.details;
    throw errorObj;
  }

  return data;
};

// ========================================
// Authentication APIs
// ========================================

export const registerUser = async (email, password, name = '') => {
  const response = await fetch(`${BASE_URL}/register`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password, name })
  });

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.error || 'Registration failed');
  }

  if (data.token) {
    setToken(data.token);
    if (data.user) setStoredUser(data.user);
  }
  return data;
};

export const loginUser = async (email, password) => {
  const response = await fetch(`${BASE_URL}/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.error || 'Login failed');
  }
```

```
  if (data.token) {
    setToken(data.token);
    if (data.user) setStoredUser(data.user);
  }
  return data;
};

export const logoutUser = () => {
  removeToken();
  window.dispatchEvent(new CustomEvent('auth:logout'));
};

// Supplementary Problem 1: GET /me endpoint
export const getMe = async () => {
  return await authFetch('/me', { method: 'GET' });
};

// ========================================
// Protected Task APIs (JWT-protected)
// ========================================

export const getTasks = async (params = {}) => {
  const query = new URLSearchParams(params).toString();
  return await authFetch(`/tasks${query ? `?${query}` : ''}`, { method: 'GET' });
};

export const getTaskById = async (id) => {
  return await authFetch(`/tasks/${id}`, { method: 'GET' });
};

export const createTask = async (taskData) => {
  return await authFetch('/tasks', {
    method: 'POST',
    body: JSON.stringify(taskData)
  });
};

export const updateTask = async (id, taskData) => {
  return await authFetch(`/tasks/${id}`, {
    method: 'PUT',
    body: JSON.stringify(taskData)
  });
};

export const deleteTask = async (id) => {
  return await authFetch(`/tasks/${id}`, {
    method: 'DELETE'
  });
};

export const checkBackendHealth = async () => {
  try {
    const res = await fetch(`${BASE_URL}/`, { cache: 'no-store' });
    return res.ok;
```

```
  } catch (err) {
    return false;
  }
};
```

7. client/src/components/AuthModal.jsx (Login & Registration Component):

```
import { useState } from 'react';
import { loginUser, registerUser } from '../api';

function AuthModal({ isOpen, onClose, onAuthSuccess }) {
  const [isLoginMode, setIsLoginMode] = useState(true);
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [name, setName] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  if (!isOpen) return null;

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    if (!email || !password) {
      setError('Please fill in both email and password.');
      return;
    }

    if (!isLoginMode && password.length < 6) {
      setError('Password must be at least 6 characters long.');
      return;
    }

    try {
      setLoading(true);
      let result;
      if (isLoginMode) {
        result = await loginUser(email, password);
      } else {
        result = await registerUser(email, password, name);
      }

      setLoading(false);
      onAuthSuccess(result.user, isLoginMode ? 'Logged in successfully!' : 'Account registered and
logged in!');
      onClose();
    } catch (err) {
      setLoading(false);
      setError(err.message || 'Authentication failed. Please check credentials.');
    }
  };

  const handleFillDemo = () => {
```

```
    setEmail('24cs047@charusat.edu.in');
    setPassword('ManavLakhani2026!');
    setName('Manav Lakhani (24CS047)');
  };

  return (
    <div className="modal-backdrop" onClick={onClose}>
      <div className="modal-card auth-modal-card" onClick={(e) => e.stopPropagation()}>
        <div className="modal-header">
          <div className="auth-header-title">
            <span className="auth-icon">{isLoginMode ? '🔐' : '📝'}</span>
            <h3 className="modal-title">
              {isLoginMode ? 'User Login' : 'Create Account'}
            </h3>
          </div>
          <button className="modal-close-btn" onClick={onClose} aria-label="Close dialog">
            ×
          </button>
        </div>

        <div className="auth-tabs">
          <button
            type="button"
            className={`auth-tab-btn ${isLoginMode ? 'active' : ''}`}
            onClick={() => {
              setIsLoginMode(true);
              setError('');
            }}
          >
            Login
          </button>
          <button
            type="button"
            className={`auth-tab-btn ${!isLoginMode ? 'active' : ''}`}
            onClick={() => {
              setIsLoginMode(false);
              setError('');
            }}
          >
            Register
          </button>
        </div>

        {error && (
          <div className="auth-error-alert">
            <span>⚠️</span> {error}
          </div>
        )}

        <form onSubmit={handleSubmit} className="auth-form">
          {!isLoginMode && (
            <div className="form-group">
              <label htmlFor="auth-name">Full Name (Optional)</label>
              <input
```

```
                id="auth-name"
                type="text"
                className="form-control"
                placeholder="e.g. Manav Lakhani"
                value={name}
                onChange={(e) => setName(e.target.value)}
              />
            </div>
          )}

          <div className="form-group">
            <label htmlFor="auth-email">Email Address</label>
            <input
              id="auth-email"
              type="email"
              className="form-control"
              placeholder="e.g. user@charusat.edu.in"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              required
              autoFocus
            />
          </div>

          <div className="form-group">
            <label htmlFor="auth-password">Password (min 6 characters)</label>
            <input
              id="auth-password"
              type="password"
              className="form-control"
              placeholder="••••••••"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              required
            />
          </div>

          <div className="auth-actions">
            <button
              type="button"
              className="btn btn-secondary demo-fill-btn"
              onClick={handleFillDemo}
            >
              Fill Demo Credentials
            </button>
            <button
              type="submit"
              className="btn btn-primary auth-submit-btn"
              disabled={loading}
            >
              {loading ? 'Processing...' : isLoginMode ? 'Login' : 'Register & Login'}
            </button>
          </div>
        </form>
```

```
        <div className="auth-footer-note">
          <small>
            Practical 7 Authentication & Middleware Pipeline • Passwords hashed with bcrypt (salt
rounds: 10) • JWT with 1h expiry
          </small>
        </div>
      </div>
    </div>
  );
}

export default AuthModal;
```

8. client/src/components/TasksPage.jsx (Protected Task Management Dashboard):

```
import { useState, useEffect, useCallback } from 'react';
import { getTasks, createTask, updateTask, deleteTask, checkBackendHealth } from '../api';
import TaskForm from './TaskForm';
import TaskList from './TaskList';
import TaskEditModal from './TaskEditModal';
import ConfirmModal from './ConfirmModal';
import Toast from './Toast';
import Spinner from './Spinner';
import ErrorMessage from './ErrorMessage';

function TasksPage({ currentUser, onOpenAuthModal, onLogout }) {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [backendOnline, setBackendOnline] = useState(true);

  // Filters & Search
  const [statusFilter, setStatusFilter] = useState('all'); // 'all', 'pending', 'completed'
  const [searchQuery, setSearchQuery] = useState('');

  // Supplementary Problem 1: Optimistic UI toggle
  const [useOptimisticUi, setUseOptimisticUi] = useState(true);

  // Active operation loading states
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [isUpdatingId, setIsUpdatingId] = useState(null);

  // Supplementary Problem 2: Delete confirmation modal
  const [deleteModalTask, setDeleteModalTask] = useState(null);
  const [isDeleting, setIsDeleting] = useState(false);

  // Edit modal state
  const [editModalTask, setEditModalTask] = useState(null);
  const [isSavingEdit, setIsSavingEdit] = useState(false);

  // Toast notification state
  const [toast, setToast] = useState(null);

  const showToast = (message, type = 'success') => {
```

```
    setToast({ message, type });
  };

  // Fetch tasks for authenticated user
  const fetchTaskList = useCallback(async (isInitial = false) => {
    if (!currentUser) return;

    if (isInitial) setLoading(true);
    setError(null);
    try {
      const isHealthy = await checkBackendHealth();
      setBackendOnline(isHealthy);
      if (!isHealthy) {
        throw new Error('Express backend on port 5000 is unreachable. Ensure the server is
running.');
      }

      const res = await getTasks();
      setTasks(res.data || []);
    } catch (err) {
      if (err.status === 401) {
        // Handled via global listener
        return;
      }
      setError(err.message || 'Failed to communicate with MongoDB Atlas backend');
      showToast(err.message || 'Failed to fetch tasks', 'error');
    } finally {
      setLoading(false);
    }
  }, [currentUser]);

  useEffect(() => {
    if (currentUser) {
      fetchTaskList(true);
    } else {
      setTasks([]);
    }
  }, [currentUser, fetchTaskList]);

  // Handle Create Task (with Optimistic UI support)
  const handleCreateTask = async (taskData) => {
    if (!currentUser) {
      onOpenAuthModal();
      return false;
    }

    setIsSubmitting(true);

    const tempId = 'temp-' + Date.now();
    let rollbackTasks = null;

    if (useOptimisticUi) {
      rollbackTasks = [...tasks];
      const optimisticTask = {
        _id: tempId,
```

```
        ...taskData,
        createdAt: new Date().toISOString(),
        isOptimistic: true
      };
      setTasks([optimisticTask, ...tasks]);
      showToast('Task added optimistically. Syncing to MongoDB Atlas...', 'success');
    }

    try {
      const res = await createTask(taskData);
      if (useOptimisticUi) {
        setTasks((prev) =>
          prev.map((t) => (t._id === tempId ? res.data : t))
        );
      } else {
        setTasks((prev) => [res.data, ...prev]);
      }
      showToast('Task successfully saved to MongoDB Atlas database!', 'success');
      return true;
    } catch (err) {
      if (useOptimisticUi && rollbackTasks) {
        setTasks(rollbackTasks);
        showToast(`Sync Failed: ${err.message}. Task rolled back.`, 'error');
      } else {
        showToast(err.message || 'Failed to create task', 'error');
      }
      return false;
    } finally {
      setIsSubmitting(false);
    }
  };

  // Handle Toggle Task Completion
  const handleToggleComplete = async (task) => {
    const updatedStatus = !task.completed;
    const previousTasks = [...tasks];

    // Optimistic toggle
    setTasks((prev) =>
      prev.map((t) => (t._id === task._id ? { ...t, completed: updatedStatus } : t))
    );
    setIsUpdatingId(task._id);

    try {
      await updateTask(task._id, { completed: updatedStatus });
      showToast(
        updatedStatus ? `Task marked as completed!` : `Task marked as pending!`,
        'success'
      );
    } catch (err) {
      setTasks(previousTasks);
      showToast(`Update Failed: ${err.message}. Reverting status.`, 'error');
    } finally {
      setIsUpdatingId(null);
    }
```

```
  };

  // Open Delete Confirmation Modal
  const handleRequestDelete = (task) => {
    setDeleteModalTask(task);
  };

  // Confirm Delete Action
  const handleConfirmDelete = async () => {
    if (!deleteModalTask) return;
    const taskToDelete = deleteModalTask;
    setIsDeleting(true);

    try {
      await deleteTask(taskToDelete._id);
      setTasks((prev) => prev.filter((t) => t._id !== taskToDelete._id));
      showToast(`Task "${taskToDelete.title}" deleted from database.`, 'success');
      setDeleteModalTask(null);
    } catch (err) {
      showToast(`Delete Failed: ${err.message}`, 'error');
    } finally {
      setIsDeleting(false);
    }
  };

  // Open Edit Modal
  const handleRequestEdit = (task) => {
    setEditModalTask(task);
  };

  // Save Edit Changes
  const handleSaveEdit = async (id, updatedFields) => {
    setIsSavingEdit(true);
    try {
      const res = await updateTask(id, updatedFields);
      setTasks((prev) => prev.map((t) => (t._id === id ? res.data : t)));
      showToast('Task updated successfully!', 'success');
      setEditModalTask(null);
    } catch (err) {
      showToast(`Update Failed: ${err.message}`, 'error');
    } finally {
      setIsSavingEdit(false);
    }
  };

  // Filter & Search Logic
  const filteredTasks = tasks.filter((task) => {
    if (statusFilter === 'pending' && task.completed) return false;
    if (statusFilter === 'completed' && !task.completed) return false;
    if (searchQuery.trim()) {
      const q = searchQuery.toLowerCase();
      const matchTitle = task.title?.toLowerCase().includes(q);
      const matchDesc = task.description?.toLowerCase().includes(q);
      if (!matchTitle && !matchDesc) return false;
    }
```

```
    return true;
  });

  const totalCount = tasks.length;
  const completedCount = tasks.filter((t) => t.completed).length;
  const pendingCount = totalCount - completedCount;

  return (
    <div className="tasks-page-container">
      {/* Toast Alert */}
      {toast && (
        <Toast
          message={toast.message}
          type={toast.type}
          onClose={() => setToast(null)}
        />
      )}

      {/* Hero Header */}
      <div className="tasks-hero-header">
        <div className="tasks-hero-meta">
          <span className="badge-pill">Practical 7 • Authentication & Middleware Pipeline</span>
          <h1 className="tasks-page-title">Full-Stack Task Dashboard</h1>
          <p className="tasks-page-subtitle">
            Secure full-stack task management with JWT Bearer tokens, bcrypt hashing, input
validation pipeline, and MongoDB Atlas.
          </p>
        </div>

        {/* Server & Database Diagnostics */}
        <div className="backend-status-card">
          <div className="status-indicator">
            <span className={`status-dot ${backendOnline ? 'online' : 'offline'}`} />
            <span className="status-label">
              Backend (Port 5000): {backendOnline ? 'Online (CORS Enabled)' : 'Offline'}
            </span>
          </div>
          <div className="atlas-indicator">
            <span className="atlas-icon">🍃</span>
            <span className="atlas-text">MongoDB Atlas: Connected (task_manager_db)</span>
          </div>
        </div>
      </div>

      {/* Auth Banner & Controls */}
      <div className="auth-status-bar">
        {currentUser ? (
          <div className="auth-active-info">
            <div className="auth-user-identity">
              <span className="auth-avatar">👤</span>
              <div>
                <strong>{currentUser.name || 'Student User'}</strong>
                <span className="auth-email-tag">{currentUser.email}</span>
              </div>
            </div>
```

```
            <div className="auth-session-details">
              <span className="jwt-badge">🛡️ JWT Active (1h)</span>
              <button
                type="button"
                className="btn btn-secondary auth-logout-btn"
                onClick={onLogout}
              >
                Log Out
              </button>
            </div>
          </div>
        ) : (
          <div className="auth-prompt-card">
            <div className="auth-prompt-content">
              <span className="lock-icon">🔒</span>
              <div>
                <h3>Authentication Required for Task Access</h3>
                <p>
                  Task routes are protected by Express JWT middleware. Please log in or register
an account to view and manage your tasks.
                </p>
              </div>
            </div>
            <div className="auth-prompt-actions">
              <button
                type="button"
                className="btn btn-primary auth-open-btn"
                onClick={onOpenAuthModal}
              >
                🔐 Login / Register
              </button>
            </div>
          </div>
        )}
      </div>

      {/* If Authenticated: Show Tasks Dashboard */}
      {currentUser && (
        <>
          {/* Controls Bar: Optimistic UI Toggle + Search + Status Filter */}
          <div className="tasks-controls-panel">
            <div className="optimistic-toggle-box">
              <label className="toggle-switch-label">
                <input
                  type="checkbox"
                  checked={useOptimisticUi}
                  onChange={(e) => setUseOptimisticUi(e.target.checked)}
                />
                <span className="toggle-slider" />
                <span className="toggle-text">
                  Optimistic UI: <strong>{useOptimisticUi ? 'ENABLED' : 'DISABLED'}</strong>
                </span>
              </label>
              <small className="toggle-hint">
```

```
              {useOptimisticUi
                ? 'New tasks render immediately while syncing in background'
                : 'UI waits for MongoDB Atlas response before rendering'}
            </small>
          </div>

          <div className="search-filter-box">
            <input
              type="text"
              placeholder="🔍 Search tasks by title or notes..."
              className="search-input"
              value={searchQuery}
              onChange={(e) => setSearchQuery(e.target.value)}
            />

            <div className="filter-buttons">
              <button
                type="button"
                className={`filter-btn ${statusFilter === 'all' ? 'active' : ''}`}
                onClick={() => setStatusFilter('all')}
              >
                All ({totalCount})
              </button>
              <button
                type="button"
                className={`filter-btn ${statusFilter === 'pending' ? 'active' : ''}`}
                onClick={() => setStatusFilter('pending')}
              >
                Pending ({pendingCount})
              </button>
              <button
                type="button"
                className={`filter-btn ${statusFilter === 'completed' ? 'active' : ''}`}
                onClick={() => setStatusFilter('completed')}
              >
                Completed ({completedCount})
              </button>
            </div>
          </div>
        </div>

        {/* Form and List Grid */}
        <div className="tasks-main-grid">
          <div className="grid-form-column">
            <TaskForm
              onSubmit={handleCreateTask}
              isSubmitting={isSubmitting}
            />
          </div>

          <div className="grid-list-column">
            {loading ? (
              <div className="loading-card">
                <Spinner />
                <p>Loading tasks from MongoDB Atlas...</p>
```

```
                </div>
              ) : error ? (
                <ErrorMessage
                  message={error}
                  onRetry={() => fetchTaskList(true)}
                />
              ) : (
                <TaskList
                  tasks={filteredTasks}
                  onToggleComplete={handleToggleComplete}
                  onRequestDelete={handleRequestDelete}
                  onRequestEdit={handleRequestEdit}
                  isUpdatingId={isUpdatingId}
                />
              )}
            </div>
          </div>
        </>
      )}

      {/* Confirmation Modal */}
      <ConfirmModal
        isOpen={Boolean(deleteModalTask)}
        taskTitle={deleteModalTask?.title}
        isDeleting={isDeleting}
        onConfirm={handleConfirmDelete}
        onCancel={() => setDeleteModalTask(null)}
      />

      {/* Edit Modal */}
      <TaskEditModal
        isOpen={Boolean(editModalTask)}
        task={editModalTask}
        isSaving={isSavingEdit}
        onSave={handleSaveEdit}
        onCancel={() => setEditModalTask(null)}
      />
    </div>
  );
}

export default TasksPage;
```

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_7

Output

1. Password Hashing with bcryptjs (Step 3):
Implemented secure password hashing using bcryptjs with a cost factor of 10 prior to storing user credentials in MongoDB Atlas. Plaintext passwords are never stored in the database, ensuring resilience against brute-force and dictionary attacks.

2. JWT Generation & Expiration (Step 4):
Implemented JSON Web Token (JWT) signing using process.env.JWT_SECRET on both registration (POST /register) and login (POST /login). Set token expiration to 1 hour (expiresIn: "1h"). Payload embeds the user ID and email.

3. Authentication Middleware Pipeline (Step 5 & 6):
Created authMiddleware (middleware/auth.js) which extracts and validates the token from the Authorization header using the format "Bearer <token>". Handled token verification inside a try/catch block, returning structured 401 Unauthorized errors on missing, invalid, or expired tokens without crashing the process.

4. Server-Side Input Validation Pipeline (Step 7):
Implemented request validation middleware (middleware/validate.js) enforcing required fields before request data reaches controllers. Rejects invalid email formats, passwords shorter than 6 characters, empty task titles, invalid priority enums, and malformed 24-character ObjectIds with 400 Bad Request responses.

5. Supplementary Problem 1 - /me Endpoint:
Added protected endpoint GET /me that uses the decoded JWT payload to retrieve the authenticated user profile from MongoDB Atlas, excluding the hashed password from the response.

6. Supplementary Problem 2 - Frontend 401 Token Expiry Handling:
Configured centralized fetch wrapper in client/src/api.js that intercepts 401 Unauthorized status codes, purges expired tokens and cached user data from localStorage, dispatches an auth:unauthorized event, and prompts the user to log in again.

7. Client Logout Mechanism:
Integrated client-side logout functionality clearing the JWT token and user profile from localStorage, resetting application state and restoring protected view barriers.

8. Cloud MongoDB Atlas Persistence:
Maintained cloud MongoDB Atlas connectivity to cluster0.charusat.mongodb.net across all User and Task operations with automated pre-save hooks and user relationship scoping.

9. Automated Verification Test Suite (16 / 16 Tests Passed - 100%):
Executed automated integration tests covering the complete authentication lifecycle, invalid credentials, malformed/missing headers, protected route enforcement, input validation, and full CRUD operations.

Authentication & Middleware Pipeline Verification Test Log:

```
============================================================
  Practical 7: JWT Authentication & Middleware Test Suite
  Student: Manav Lakhani (24CS047) | Course: ITUE301
============================================================

[MongoDB Atlas] Connected successfully to: cluster0.charusat.mongodb.net
[PASS] Test 1: GET / returns 200 OK with API discovery manifest
```

```
[PASS] Test 2: POST /register with invalid email returns 400 Bad Request
[PASS] Test 3: POST /register with short password returns 400 Bad Request
[PASS] Test 4: POST /register registers user with bcrypt and returns 201 + JWT
[PASS] Test 5: POST /register with duplicate email rejected with 400
[PASS] Test 6: POST /login with incorrect password rejected with 401 Unauthorized
[PASS] Test 7: POST /login validates password and returns 200 OK + JWT token
[PASS] Test 8: GET /me returns logged-in user profile from decoded token
[PASS] Test 9: GET /me without token returns 401 Unauthorized
[PASS] Test 10: GET /tasks protected route blocks unauthenticated request with 401
[PASS] Test 11: GET /tasks with tampered token safely rejected with 401
[PASS] Test 12: POST /tasks rejected by validation middleware with 400 Bad Request
[PASS] Test 13: POST /tasks creates task in MongoDB Atlas and returns 201 Created
[PASS] Test 14: GET /tasks retrieves tasks list for authenticated user
[PASS] Test 15: PUT /tasks/:id updates task with valid token and returns 200 OK
[PASS] Test 16: DELETE /tasks/:id deletes task and returns 200 OK

============================================================
  Test Results Summary: 16 / 16 Tests Passed (100%)
============================================================
```